from Beginner to Master

DevOps를 위한 Docker 가상화

Section 9. Advanced Docker Usage

프로필

Dowon Lee

수강생 19K 강의평점 4.8(1K)

멘토링 ON

멘토링 설정

- 홈
- 강의
- 로드맵
- 수강평
- 게시글
- 블로그

수정하기

이 되었던 때가 있었습니
"IT 엔지니어"라고 적고

을 가지고 있습니다. 누구
람입니다. 개발자로써, 강
지만, 그래도, 남들보다 조

tecture, Blockchain,

강의 전체 6

[개정판] 웹 애플리케이션 개발을 위한 IntelliJ IDEA 설정

학습중

0강 / 25강 (0%)

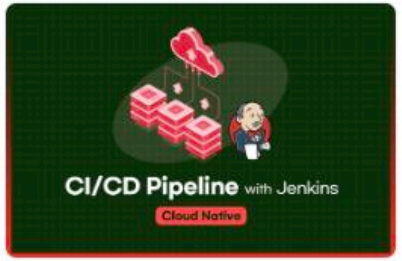
818명

멀티OS 사용을 위한 가상화 환경 구축 가이드 (Docker + Kubernetes)

학습중

0강 / 16강 (0%)

924명



Jenkins를 이용한 CI/CD Pipeline 구축

학습중

81강 / 83강 (98%)

독점 2709명

Spring Cloud로 개발하는 마이크로서비스 애플리케이션(MSA)

학습중

156강 / 156강 (100%)

독점 5531명

Spring Boot를 이용한 RESTful Web Services 개발

학습중

3강 / 48강 (6%)

독점 3862명

[구버전] 웹 애플리케이션 개발을 위한 IntelliJ IDEA 설정 (2020 ver.)

학습중

14강 / 14강 (100%)

4813명

목차

njone company

Section 9.

Advanced Docker Usage

- Docker와 MSA
- 멀티 플랫폼 이미지를 위한 Docker Buildx
- Docker Runtime을 대신하는 기술
- Docker Registry를 대신하는 기술

njone company

Cloud Native Architecture

- https://www.cncf.io/blog/2018/03/08/introducing-the-cloud-native-landscape-2-0-interactive-edition/

Microservices	Containerization	Continuous delivery	DevOps
A microservices architecture is an application development approach in which a large application is built as a suite of modular components or services	Containers are a type of software that can virtually package and isolate applications for deployment	Continuous delivery is a software delivery approach in which development teams produce and test code in short but continuous cycles	DevOps is a method-ology that promotes better communication and collaboration between development and operations teams

njone company

Microservices

Microservices

A microservices architecture is an application development approach in which a large application is built as a suite of modular components or services

Containerization

Containers are a type of software that can virtually package and isolate applications for deployment

Continuous delivery

Continuous delivery is a software delivery approach in which development teams produce and test code in short but continuous cycles

DevOps

DevOps is a method-ology that promotes better communication and collaboration between development and operations teams

njone company

Containerization

Microservices

A microservices architecture is an application development approach in which a large application is built as a suite of modular components or services

Containerization

Containers are a type of software that can virtually package and isolate applications for deployment

Continuous delivery

Continuous delivery is a software delivery approach in which development teams produce and test code in short but continuous cycles

DevOps

DevOps is a method-ology that promotes better communication and collaboration between development and operations teams

njone company

Continuous delivery

Microservices

A microservices architecture is an application development approach in which a large application is built as a suite of modular components or services

Containerization

Containers are a type of software that can virtually package and isolate applications for deployment

Continuous delivery

Continuous delivery is a software delivery approach in which development teams produce and test code in short but continuous cycles

DevOps

DevOps is a method-ology that promotes better communication and collaboration between development and operations teams

njone company

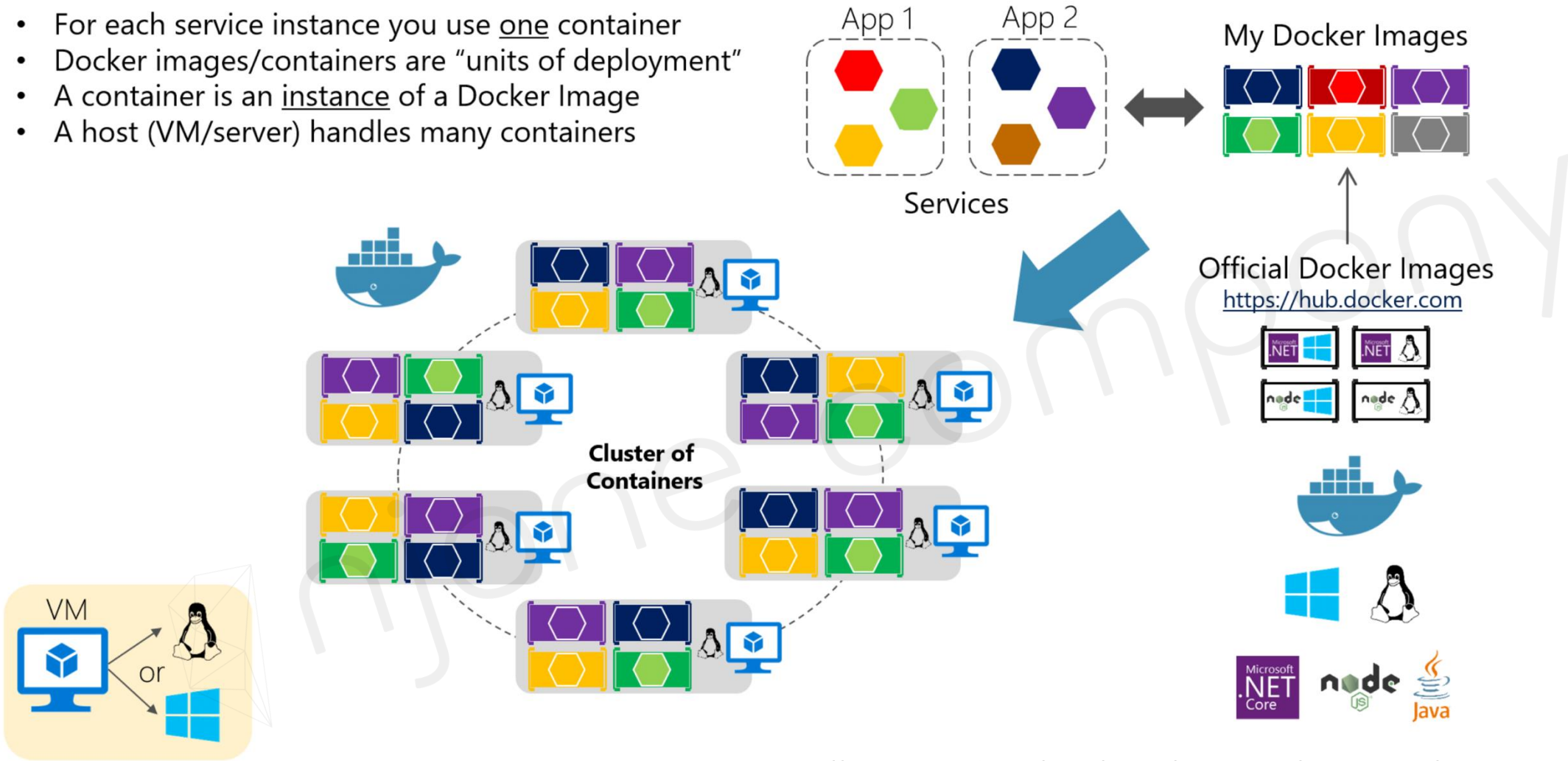
DevOps

Microservices	Containerization	Continuous delivery	DevOps
A microservices architecture is an application development approach in which a large application is built as a suite of modular components or services	Containers are a type of software that can virtually package and isolate applications for deployment	Continuous delivery is a software delivery approach in which development teams produce and test code in short but continuous cycles	DevOps is a method-ology that promotes better communication and collaboration between development and operations teams

njone company

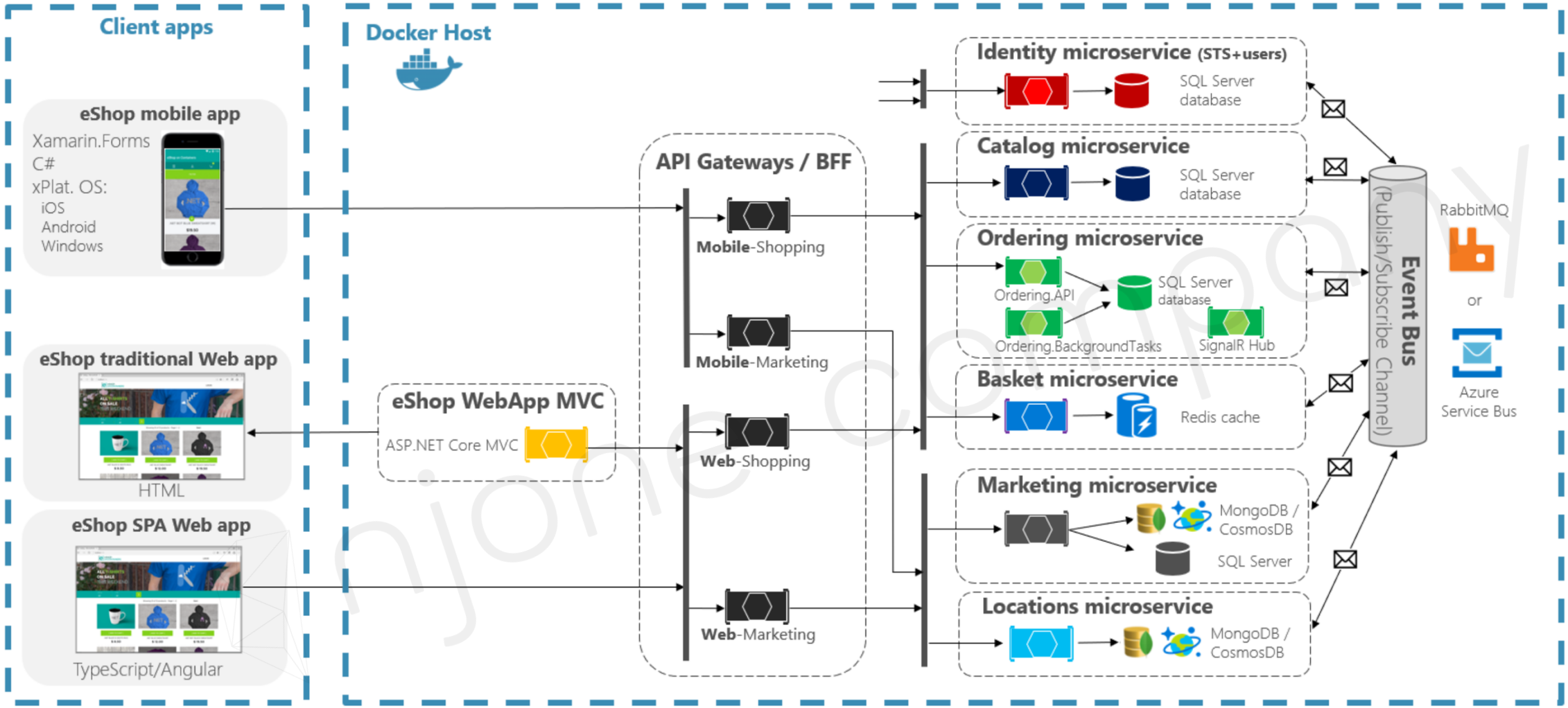
Composed Docker Applications in a Cluster

- For each service instance you use one container
- Docker images/containers are "units of deployment"
- A container is an instance of a Docker Image
- A host (VM/server) handles many containers



https://learn.microsoft.com/en-us/dotnet/architecture/microservices/architect-microservice-container-applications/scalable-available-multi-container-microservice-applications

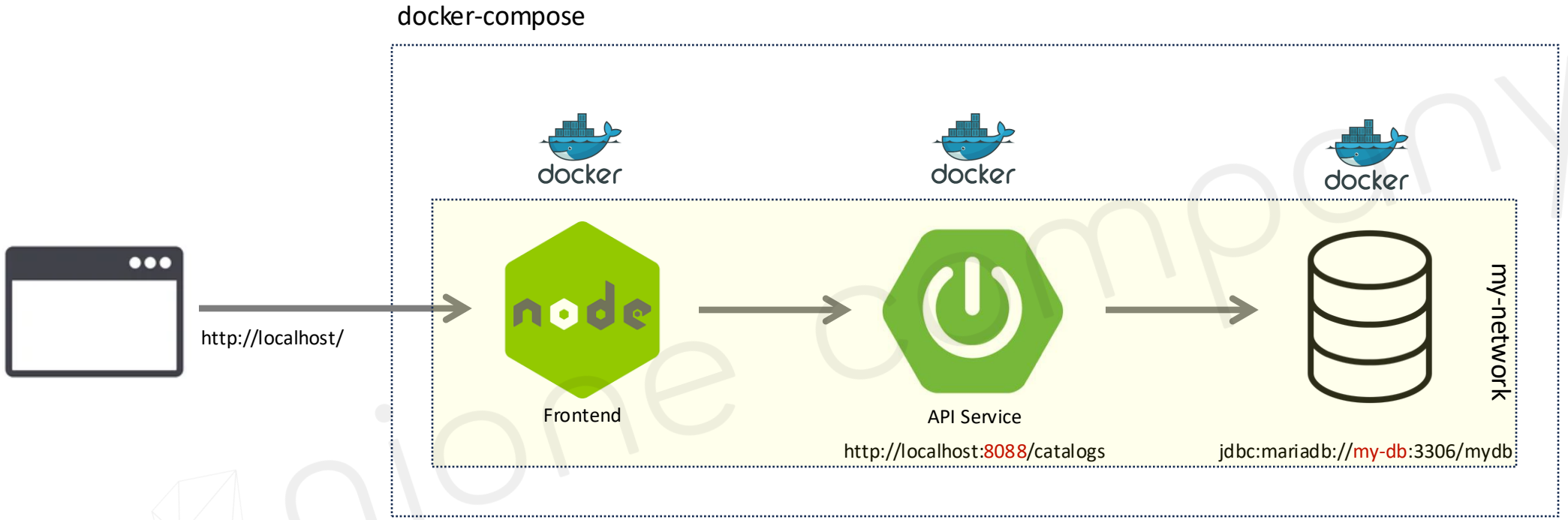
Development environment architecture



https://codeboulevard.com/2023/11/microservice-roadmap/

Docker와 MSA

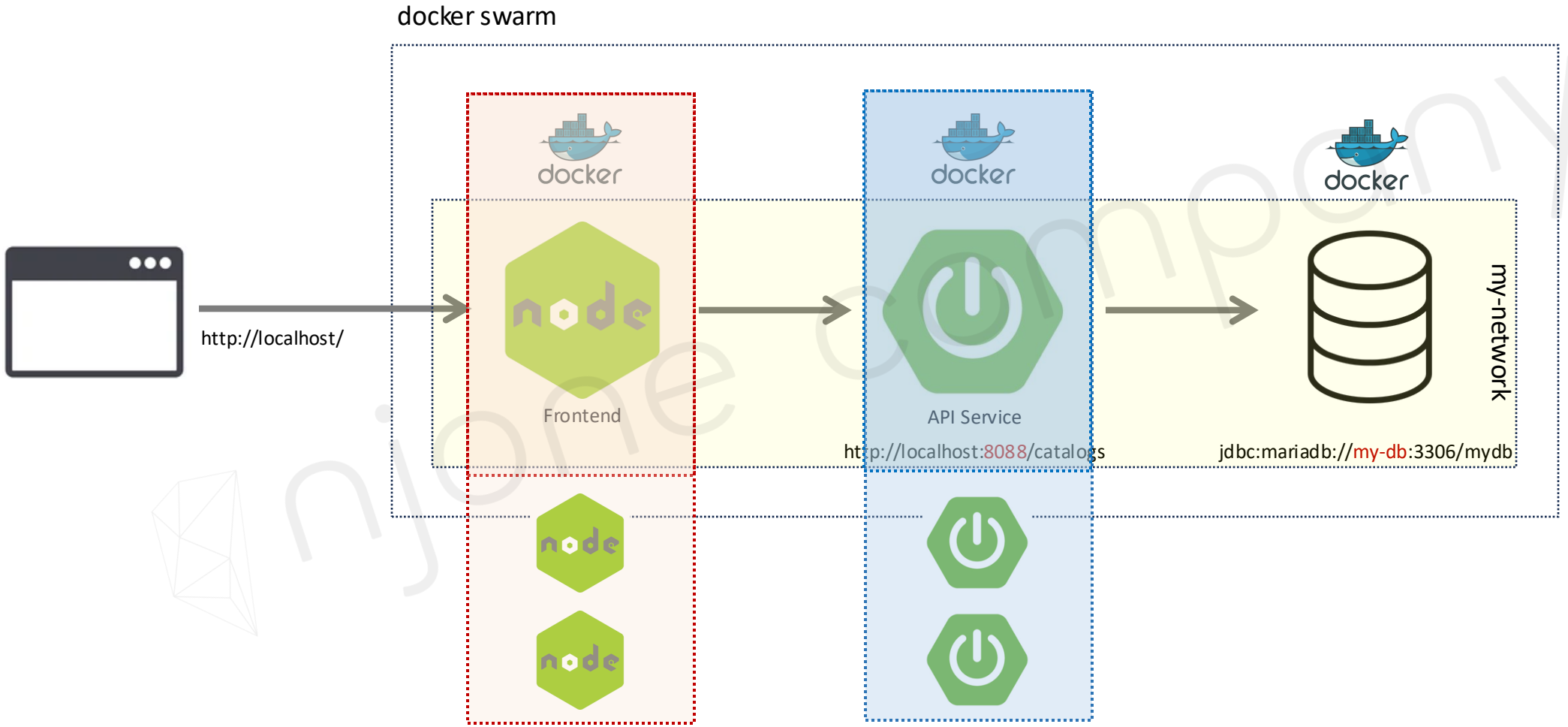
MSA + Docker



njone company

Docker와 MSA

MSA + Docker



njone company

멀티 플랫폼 이미지를 위한 Docker Buildx

Docker Buildx?

- Multi Platform 빌드 기능 지원
  - Docker 19.03 부터 사용 가능
  - Docker Desktop → Buildx 플러그인 포함

- 지원 가능한 Platform

$ docker buildx ls

```
$ docker buildx ls
NAME/NODE           DRIVER/ENDPOINT     STATUS    BUILDKIT   PLATFORMS
default             docker
 \_ default          \_ default         running   v0.13.1    linux/arm64, linux/amd64, linux/amd64/v2, linux/riscv64,
 linux/ppc64le, linux/s390x, linux/386, linux/mips64le, linux/mips64, linux/arm/v7, linux/arm/v6
desktop-linux*      docker
 \_ desktop-linux    \_ desktop-linux   running   v0.13.1    linux/arm64, linux/amd64, linux/amd64/v2, linux/riscv64,
 linux/ppc64le, linux/s390x, linux/386, linux/mips64le, linux/mips64, linux/arm/v7, linux/arm/v6
```

```
$ docker buildx
Extended build capabilities with BuildKit

Usage:  docker buildx [OPTIONS] COMMAND

Extended build capabilities with BuildKit

Options:
      --builder string   Override the configured builder instance

Management Commands:
  imagetools  Commands to work on images in registry

Commands:
                                                    ce
                                                    to builder instance
                                                    tance
  rm          Remove one or more builder instances
  stop        Stop builder instance
  use         Set the current builder instance
  version     Show buildx version information
```

멀티 플랫폼 이미지를 위한 Docker Buildx

Cross-Platform Build

- MacOS Apple chip
  - Linux 이미지 빌드

```
FROM node:alpine
WORKDIR /app
```

```
$ docker build --tag edowon0623/my-node:m1 -f Dockerfile .
```

```
docker image inspect edowon0623/my-node:m1 | grep Architecture
        "Architecture": "arm64",
```

```
$ docker push edowon0623/my-node:m1
```

njone company

Cross-Platform Build

- Linux 1)
  - Linux 이미지 사용

$ docker pull edowon0623/my-node:m1

```
[ec2-user@ip-172-31-5-111 ~]$ docker images | grep my-node
edowon0623/my-node          m1                  45b0e0a5bf15   6 minutes ago   137MB
```

$ docker run -it edowon0623/my-node:m1 bash

```
[ec2-user@ip-172-31-5-111 ~]$ docker run -it edowon0623/my-node:m1 bash
WARNING: The requested image's platform (linux/arm64/v8) does not match the detected host platform (linux/amd64) and
 no specific platform was requested
exec /usr/local/bin/docker-entrypoint.sh: exec format error
```

njone company

멀티 플랫폼 이미지를 위한 Docker Buildx

Cross-Platform Build

- MacOS Apple chip
  - Linux 이미지 Buildx 빌드

$ docker buildx imagetools inspect node:alpine

```
docker buildx imagetools inspect node:alpine
Name:      docker.io/library/node:alpine
MediaType: application/vnd.docker.distribution.manifest.list.v2+json
Digest:    sha256:db8772d9f5796ac4e8c47508038c413ea1478da010568a2e48672f19a8b80cd2

Manifests:
  Name:      docker.io/library/node:alpine@sha256:c56ef510b1a041de46939462d89312015465895a6255434d012f1d2dcc8d85c7
  MediaType: application/vnd.docker.distribution.manifest.v2+json
  Platform:  linux/amd64

  Name:      docker.io/library/node:alpine@sha256:b84f96382d6a3b6b54add6172cf9627467e2b8b58e007471415773ed4d96c9c7
  MediaType: application/vnd.docker.distribution.manifest.v2+json
  Platform:  linux/arm/v6

  Name:      docker.io/library/node:alpine@sha256:58943268b3df5849ad18b8bfac735f8a30a35b4104a5269255a8adaacd168e72
  MediaType: application/vnd.docker.distribution.manifest.v2+json
  Platform:  linux/arm/v7

  Name:      docker.io/library/node:alpine@sha256:4233aea25ab7f7d288b23a0a80644da9fea41c4772ac4562840281e55c4ce54d
  MediaType: application/vnd.docker.distribution.manifest.v2+json
  Platform:  linux/arm64/v8
```

멀티 플랫폼 이미지를 위한 Docker Buildx

Cross-Platform Build

- MacOS Apple chip
  - Linux 이미지 Buildx 빌드

```
$ docker buildx build --platform linux/amd64 --tag edowon0623/my-node:amd63 -f Dockerfile .
```

```
> docker buildx build --platform linux/amd64 --tag edowon0623/my-node:amd64 -f Dockerfile .
[+] Building 1.0s (6/6) FINISHED                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                               0.0s
 => => transferring dockerfile: 66B                                                                0.0s
 => [internal] load metadata for docker.io/library/node:alpine                                     0.9s
 => [internal] load .dockerignore                                                                  0.0s
 => => transferring context: 2B                                                                    0.0s
 => [1/2] FROM docker.io/library/node:alpine@sha256:db8772d9f5796ac4e8c47508038c413ea1478da010568a2e48672f19a8  0.0s
 => CACHED [2/2] WORKDIR /app                                                                      0.0s
 => exporting to image                                                                             0.0s
 => => exporting layers                                                                            0.0s
 => => writing image sha256:8a3055aa4e6d6e658db532d343fa1d49a77d2e80020bc3cef75c8c894ee4972d       0.0s
 => => naming to docker.io/edowon0623/my-node:amd64                                                0.0s

Build multi-platform images faster with Docker Build Cloud: https://docs.docker.com/go/docker-build-cloud
```

멀티 플랫폼 이미지를 위한 Docker Buildx

Cross-Platform Build

- MacOS Apple chip
  - Linux 이미지 (amd64) 사용

```
> docker image inspect edowon0623/my-node:amd64 ¦ grep Architecture
        "Architecture": "amd64",
```

```
> docker run -it edowon0623/my-node:amd64 bash
WARNING: The requested image's platform (linux/amd64) does not match the detected host platform (linux/arm64/v8) and
no specific platform was requested
node:internal/modules/cjs/loader:1145
  throw err;
  ^

Error: Cannot find module '/app/bash'
    at Module._resolveFilename (node:internal/modules/cjs/loader:1142:15)
    at Module._load (node:internal/modules/cjs/loader:983:27)
    at Function.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:142:12)
    at node:internal/main/run_main_module:28:49 {
  code: 'MODULE_NOT_FOUND',
  requireStack: []
}

Node.js v21.7.3
```

$ docker push edowon0623/my-node:amd64

Cross-Platform Build

- Linux 2)
  - Linux 이미지 사용

$ docker pull edowon0623/my-node:amd64

```
[ec2-user@ip-172-31-5-111 ~]$ docker images | grep my-node
edowon0623/my-node          amd64                8a3055aa4e6d   7 minutes ago    139MB
edowon0623/my-node          m1                   45b0e0a5bf15   20 minutes ago   137MB
```

$ docker run -it edowon0623/my-node:amd64 bash

```
[ec2-user@ip-172-31-5-111 ~]$ docker run -it edowon0623/my-node:amd64 sh
/app # node --version
v21.7.3
/app #
```

Docker Runtime을 대신하는 기술

CRI-O

- Docker의 한계점
  - Docker Server에 너무 많은 기능이 집중 → SPOF
  - Docker 명령어는 root 권한 필요

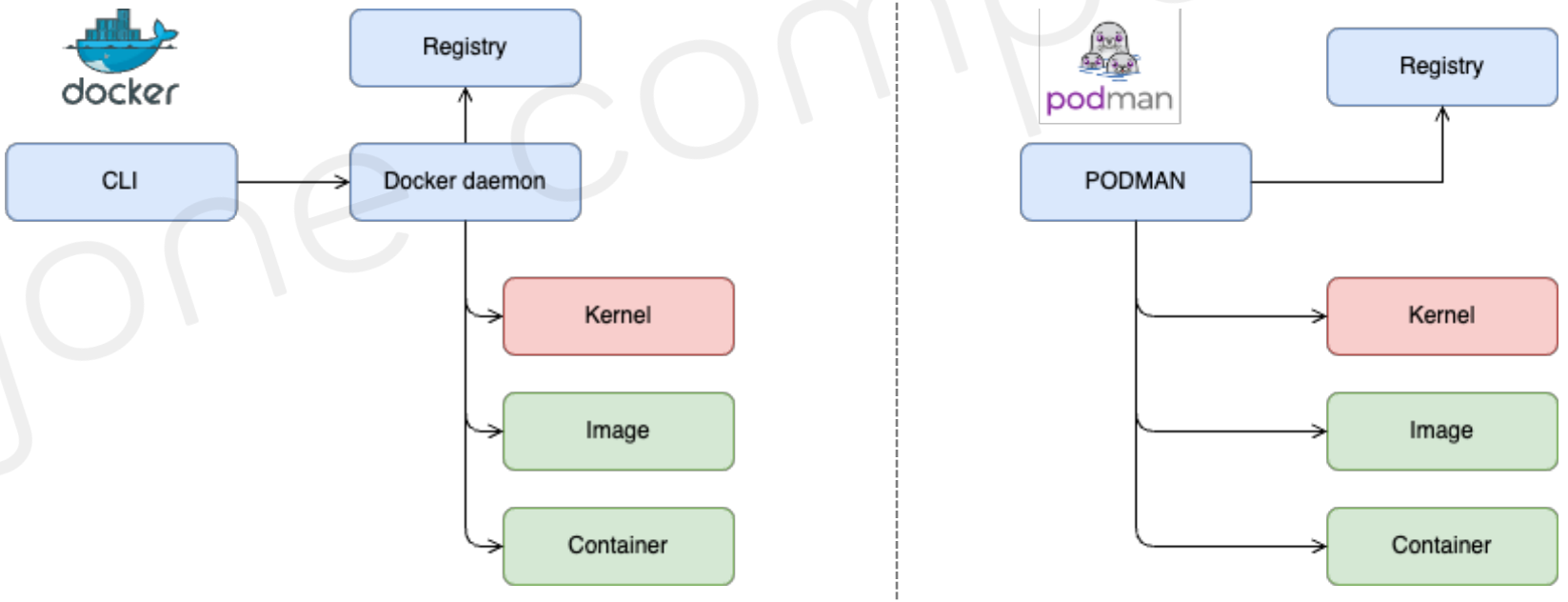
- CRI-O
  - Container Runtime Interface – Open Container Initiative
  - Container간 이식성 표준화를 위해 OCI 구성
  - Kubernetes 1.24 버전부터 Docker Runtime 제거 → Container Runtime 구성을 위해 CRI 사용
  - Container Runtime만 가능 → 이미지 빌드, CLI, Registry 기능 X
  - Container를 fork/exec. 방식으로 구동
  - Podman (Container 실행), Buildah (이미지 생성), Skopeo (이미지 저장소 관리)

njone company

Docker Runtime을 대신하는 기술

PodMan

- Docker Cli와 같은 Tool
  - Container 생성,유지 및 관리 도구
  - Container를 fork/exec. 방식으로 구동 → daemon-less
  - root 권한 필요 X



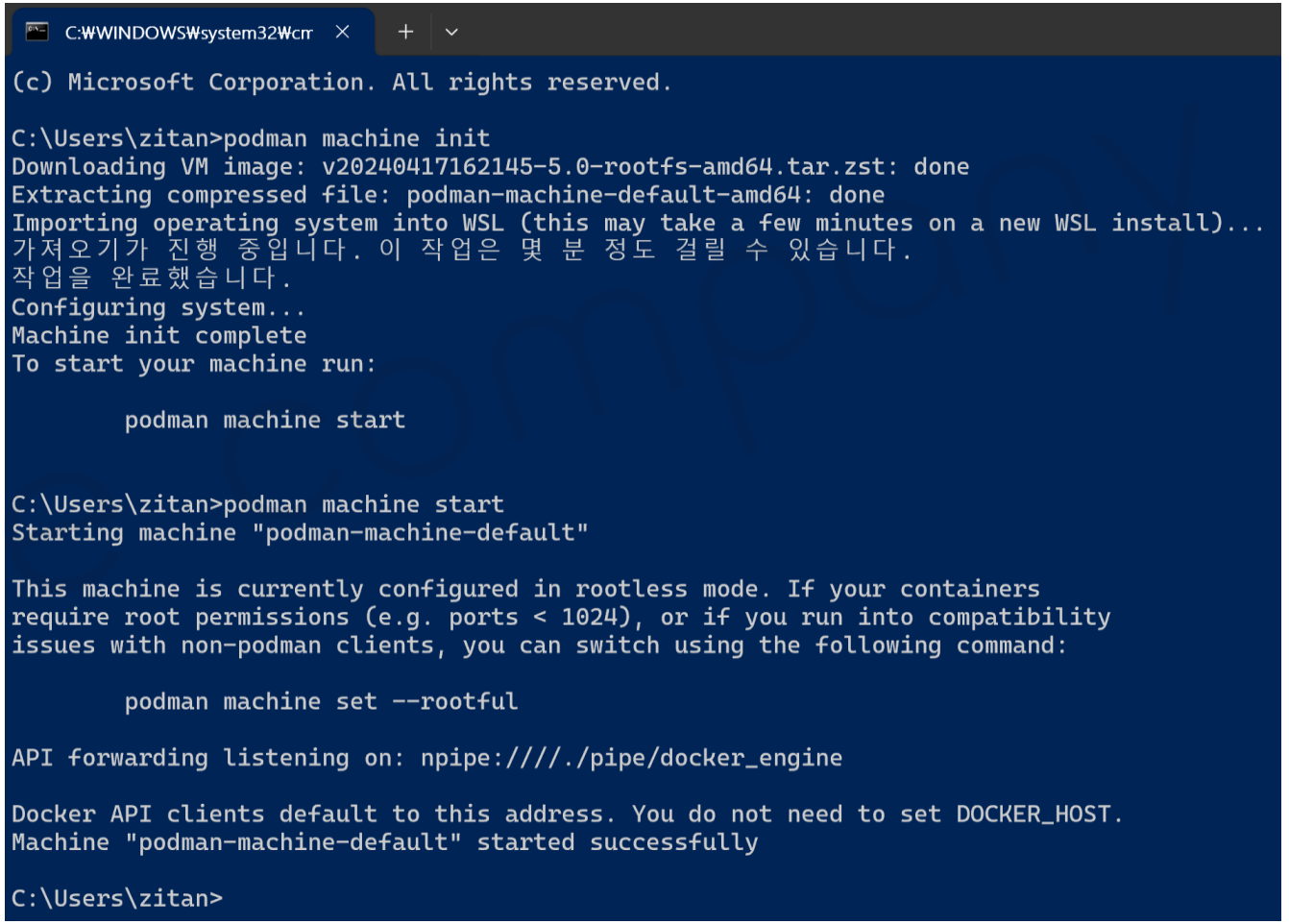
njone company

Docker Runtime을 대신하는 기술

Podman for Windows

- https://github.com/containers/podman/blob/main/docs/tutorials/podman-for-windows.md

```
C:\> podman machine init
C:\> podman machine start
C:\> podman info
```



Docker Runtime을 대신하는 기술

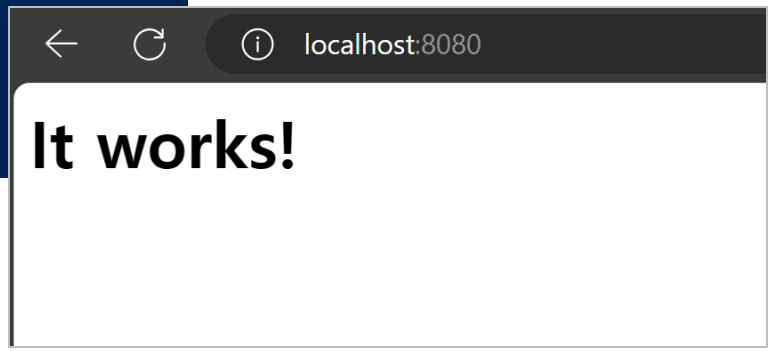
Podman for Windows

- Podman으로 Container 실행 - httpd

```
C:\> podman run --rm -d -p 8080:80 --name httpd docker.io/library/httpd
```

```
C:\Users\zitan>podman run --rm -d -p 8080:80 --name httpd docker.io/library/httpd
Trying to pull docker.io/library/httpd:latest...
Getting image source signatures
Copying blob sha256:d2b091e65160919e8aa45ccc4e2187a65be454ce437692ec811e3fe47386bd4f
Copying blob sha256:6e9a8835eae400bbb7b5572cc9aeea0a187e7216dd2def605c432cbe1d4955c5
Copying blob sha256:b927d001db705e0c4192a7c5ea2ca65c7c0f4858349d9a785e40e044d617724b
Copying blob sha256:13808c22b207b066ef43572e57e4fb8c6172e887dd9a918c089a174a19371b7a
Copying blob sha256:559cc51378ed1226b3d18ddb9d2149586a427338728bc7bd567bda03ca43b590
Copying blob sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38e8acc1
Copying config sha256:fa0099f1c09d7d1dc852123c9456a23436df95715812df3db8b960ef6609e288
Writing manifest to image destination
1c63c0153fb1ff3a70abfb3b34a03bc6f6fdccd505d405665ca05403cf91751b

C:\Users\zitan>docker images
REPOSITORY   TAG       IMAGE ID       CREATED       SIZE
httpd        latest    fa0099f1c09d   2 weeks ago   152MB
```



Docker Runtime을 대신하는 기술

Podman for Windows

- Podman으로 Container 실행 - centos

C:\> podman run -it centos

```
C:\Users\zitan>podman run -it centos
Resolved "centos" as an alias (/etc/containers/registries.conf.d/000-shortnames.conf)
Trying to pull quay.io/centos/centos:latest...
Getting image source signatures
Copying blob sha256:7a0437f04f83f084b7ed68ad9c4a4947e12fc4e1b006b38129bac89114ec3621
Copying config sha256:300e315adb2f96afe5f0b2780b87f28ae95231fe3bdd1e16b9ba606307728f55
Writing manifest to image destination
[root@47e4bb5c8e3c /]#
```

```
C:\Users\zitan>podman ps -a
CONTAINER ID  IMAGE                           COMMAND           CREATED         STATUS                     PORTS                   NAMES
1c63c0153fb1  docker.io/library/httpd:latest  httpd-foreground  3 minutes ago   Up 3 minutes               0.0.0.0:8080->80/tcp    httpd
47e4bb5c8e3c  quay.io/centos/centos:latest    /bin/bash         49 seconds ago  Exited (127) 8 seconds ago                         jovial_mcclintock
```

njone company

Docker Runtime을 대신하는 기술

Podman for MacOS (M1)

- https://podman.io/docs/installation

```
brew install qemu podman
==> Downloading https://formulae.brew.sh/api/formula.jws.json
######################################################################################################################## 100.0%
==> Downloading https://formulae.brew.sh/api/cask.jws.json
######################################################################################################################## 100.0%
==> Downloading https://ghcr.io/v2/homebrew/core/qemu/manifests/9.0.0
Already downloaded: /Users/edowon/Library/Caches/Homebrew/downloads/6d7f489eba08de6d463648d2ad0fcac872edb1733993437ad260c98a86329213-
==> Fetching qemu
```

```
==> podman
        In order to run containers locally, podman depends on a Linux kernel.
        One can be started manually using `podman machine` from this package.
        To start a podman VM automatically at login, also install the cask
        "podman-desktop".

zsh completions have been installed to:
  /usr/local/share/zsh/site-functions
(base)  edowon  ~/Desktop/Work/14.online/devops-docker  section5 ±
```

Docker Runtime을 대신하는 기술

Podman for MacOS (M1)

- https://podman.io/docs/installation

```
 podman machine init
Extracting compressed file: podman-machine-default_fedora-coreos-40.20240504.2.0-qemu.x86_64.qcow2: done
Image resized.
Machine init complete
To start your machine run:

        podman machine start

(base)  edowon  ~/Desktop/Work/14.online/devops-docker  section5 ±
```

```
 podman machine start
Starting machine "podman-machine-default"
Waiting for VM ...
Error: qemu exited unexpectedly with exit code -1, stderr: qemu-system-x86_64: Unknown Error
```

Docker Runtime을 대신하는 기술

Podman for MacOS (M3)

- https://podman.io/docs/installation

```
[redacted]-iMac ~ % brew install qemu podman
==> Downloading https://ghcr.io/v2/homebrew/core/qemu/manifests/9.0.0
##################################################################################################################
==> Fetching dependencies for qemu: capstone, dtc, pcre2, mpdecimal, ca-certificates, openssl@3, readline, sqlite, xz, pytho
p11-kit, libevent, libnghttp2, unbound, gnutls, jpeg-turbo, libpng, libslirp, libssh, libusb, lzo, ncurses, pixman, snappy,
==> Downloading https://ghcr.io/v2/homebrew/core/capstone/manifests/5.0.1
##################################################################################################################
==> Fetching capstone
==> Downloading https://ghcr.io/v2/homebrew/core/capstone/blobs/sha256:667f0e0d8960724b71c950ccc7769e747269b828c9e1c90009dc5
##################################################################################################################
==> Downloading https://ghcr.io/v2/homebrew/core/dtc/manifests/1.7.0
```

```
[redacted]-iMac ~ % podman machine init
Looking up Podman Machine image at quay.io/podman/machine-os:5.0 to create VM
Getting image source signatures
Copying blob 4532a1715d4d done   |
Copying config 44136fa355 done   |
Writing manifest to image destination
4532a1715d4d56948bae034da8c169e5fe0ee1739f8ded4268b836356e5f085d
Extracting compressed file: podman-machine-default-arm64.raw: done
Machine init complete
To start your machine run:

        podman machine start

naeun@ina-eun-ui-iMac ~ %
```

Docker Runtime을 대신하는 기술

Podman for MacOS (M3)

- https://podman.io/docs/installation

```
[redacted]-iMac ~ % podman machine start
Starting machine "podman-machine-default"

This machine is currently configured in rootless mode. If your containers
require root permissions (e.g. ports < 1024), or if you run into compatibility
issues with non-podman clients, you can switch using the following command:

        podman machine set --rootful

API forwarding listening on: /var/folders/fc/41cm3p451j51791m4sry7bpr0000gn/T/podman/podman-machine-default-api.sock

The system helper service is not installed; the default Docker API socket
address can't be used by podman. If you would like to install it, run the following commands:

        sudo /opt/homebrew/Cellar/podman/5.0.2/bin/podman-mac-helper install
        podman machine stop; podman machine start

You can still connect Docker API clients by setting DOCKER_HOST using the
following command in your terminal session:

        export DOCKER_HOST='unix:///var/folders/fc/41cm3p451j51791m4sry7bpr0000gn/T/podman/podman-machine-default-api.sock'

Machine "podman-machine-default" started successfully
```

```
[redacted]-iMac ~ % podman machine list
NAME                     VM TYPE     CREATED             LAST UP             CPUS        MEMORY      DISK SIZE
podman-machine-default*  applehv     About a minute ago  Currently running  4           2GiB        100GiB
naeun@ina-eun-ui-iMac ~ %
```

Docker Runtime을 대신하는 기술

Podman for MacOS (M3)

- https://podman.io/docs/installation

```
[redacted]-iMac ~ % podman run --rm -d -p 8080:80 --name httpd docker.io/library/httpd
Trying to pull docker.io/library/httpd:latest...
Getting image source signatures
Copying blob sha256:bae159c85a0b3662b8e9d58344f7da1b84836a7338c8ddf6b88360cea400929f
Copying blob sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38e8acc1
Copying blob sha256:22d97f6a5d13532e867231d23d92620a81874d51a456196be50154eeb32edc08
Copying blob sha256:c521cc2c4e8a439951003122cdab841fe1b1350db864e2cf9dea69298513f711
Copying blob sha256:855b568ab02c3fee1362d51176f966517b1e03ca0c8efde55eddace90835f300
Copying blob sha256:98c8bced82bd1d3557a49d39ea03bb8090bf3abc8154ab65388d9f9728f3ce02
Copying config sha256:73acb239a8eb7a9ef5b6848a5b01bb26cc03d2262056705876bf7ac6ad781f02
Writing manifest to image destination
df1fe9a8ae626d75306733ac36609b8aa40fce2b5b2490b8c91a8a496fc941f5
naeun@ina-eun-ui-iMac ~ %
```

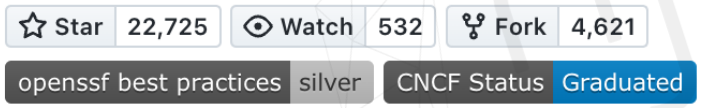
Docker Regitry를 대신하는 기술

Harbor

- https://goharbor.io/
- https://goharbor.io/docs/2.10.0/install-config/

What is Harbor?

Harbor is an open source registry that secures artifacts with policies and role-based access control, ensures images are scanned and free from vulnerabilities, and signs images as trusted. Harbor, a CNCF Graduated project, delivers compliance, performance, and interoperability to help you consistently and securely manage artifacts across cloud native compute platforms like Kubernetes and Docker.



Software	Version
Docker Engine	20.10.10-ce+ or higher
Docker Compose	1.18.0+
OpenSSL	Latest

Port	Protocol
443	HTTPS
4443	HTTPS
80	HTTP

njone.company

Docker Regitry를 대신하는 기술

인증서 설치

1) CA Certificates 생성
2) Server Certificates 생성
3) SAN(Subject Alternative Name) 등록
4) Docker Engine Certificate 업데이트

njone company

Docker Regitry를 대신하는 기술

인증서 설치

1) CA Certificates 생성

- Root CA 비밀키 생성 → 공개키 생성

```
$ openssl genrsa –out ca.key 4096
```

```
$ openssl req -x509 -new -nodes -sha512 -days 365 -key ca.key -out ca.crt
```

2) Server Certificates 생성

- 서버의 인증서를 위한 비밀키 생성 → CSR(Certificate Signing Request) 파일 생성

```
$ openssl genrsa –out server.key 4096
```

```
$ openssl req -new -sha512 -key server.key -out server.csr
```

njone company

인증서 설치

3) SAN(Subject Alternative Name) 등록

- CSR을 이용하여 SAN에 도메인 정보 추가 → 서버의 인증키 생성

```
$ vi v3ext.cnf
```

```
[root@45ad8a699362 certs]# cat v3ext.cnf
subjectAltName = IP:172.30.11.76,IP:127.0.0.1
```

Host PC IP address

```
$ openssl x509 -req -sha512 -days 365 \
    -extfile v3ext.cnf \
    -CA ca.crt -CAkey ca.key -CAcreateserial \
    -in server.csr \
    -out server.crt
```

njone company

인증서 설치

4) Docker Engine Certificate 업데이트

- Docker Engine에서 사용 될 cert 파일 생성

```
$ openssl x509 -inform PEM -in server.crt -out server.cert
```

- Docker Engine에 인증서 파일 등록

```
$ mkdir -p /etc/docker/certs.d/server
$ cp server.cert /etc/docker/certs.d/server/
$ cp server.key /etc/docker/certs.d/server/
$ cp ca.crt /etc/docker/certs.d/server/
```

njone company

Docker Regitry를 대신하는 기술

Harbor 설치

1) Harbor package 다운로드

- https://github.com/goharbor/harbor/releases

```
$ wget https://github.com/goharbor/harbor/releases/download/v2.2.2/harbor-offline-installer-v2.2.2.tgz
```

```
$ tag harbor-offline-installer-v2.2.2.tgz
```

2) harbor.yml 파일 생성 → 설정

```
$ cp harbor.yml.tmp harbor.yml
```

```
$ vi harbor.yml
```

Host PC IP address

```
hostname: 172.30.11.76

http:
  port: 80

https:
  port: 443
  certificate: /etc/docker/certs.d/server/server.cert
  private_key: /etc/docker/certs.d/server/server.key
```

njone company

Docker Regitry를 대신하는 기술

Harbor 설치

3) Deploy

```
$ ./prepare
$ ./install.sh
```

```
[root@45ad8a699362 harbor]# docker-compose ps
NAME                COMMAND                  SERVICE             STATUS              PORTS
harbor-core         "/harbor/entrypoint.…"   core                running (healthy)
harbor-db           "/docker-entrypoint.…"   postgresql          running (healthy)
harbor-jobservice   "/harbor/entrypoint.…"   jobservice          running (healthy)
harbor-log          "/bin/sh -c /usr/loc…"   log                 running (healthy)   127.0.0.1:1514->1051
harbor-portal       "nginx -g 'daemon of…"   portal              running (healthy)
nginx               "nginx -g 'daemon of…"   proxy               running (healthy)   0.0.0.0:80->8080/tcp
redis               "redis-server /etc/r…"   redis               running (healthy)
registry            "/home/harbor/entryp…"   registry            running (healthy)
registryctl         "/home/harbor/start.…"   registryctl         running (healthy)
```

Docker Regitry를 대신하는 기술

Harbor 사용

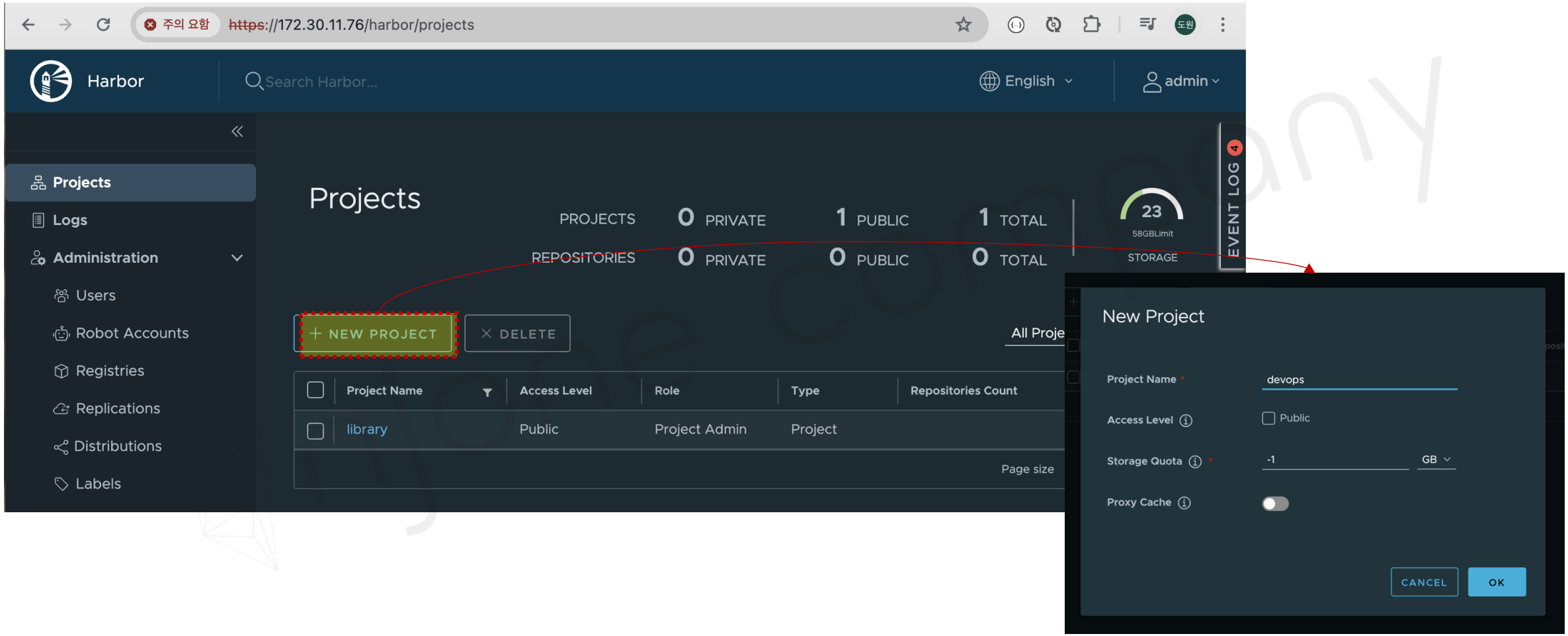
1) Project 생성
2) *Container#1]* docker login –u admin <Harbor IP address>
3) *Container#1]* docker push <IMAGE>
4) *Container#2]* docker login –u user1 <Harbor IP address>
5) *Container#2]* docker pull <IMAGE>

njone company

Docker Regitry를 대신하는 기술

Harbor 사용

1) Project 생성



Docker Regitry를 대신하는 기술

Harbor 사용

2) *Container#1]* docker login –u admin <Harbor IP address>

```
[root@45ad8a699362 harbor]# docker login -u admin 172.30.11.76
Password:
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
[root@45ad8a699362 harbor]#
```

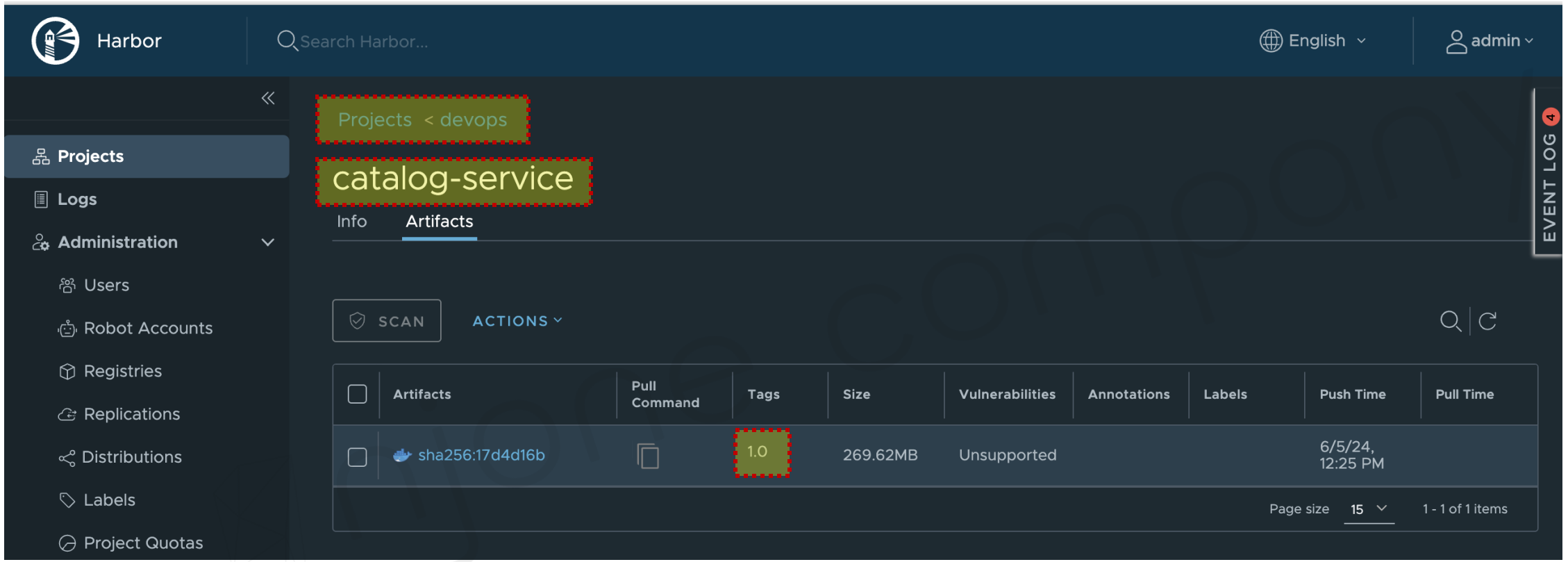
3) *Container#1]* docker push <IMAGE>

```
[root@45ad8a699362 harbor]# docker tag edowon0623/catalog-service:1.0 172.30.11.76/devops/catalog-service:1.0
[root@45ad8a699362 harbor]#
[root@45ad8a699362 harbor]# docker push 172.30.11.76/devops/catalog-service:1.0
The push refers to repository [172.30.11.76/devops/catalog-service]
b8029593adec: Layer already exists
3d3fdb9815af: Layer already exists
08664b16f94c: Layer already exists
9eb82f04c782: Layer already exists
1.0: digest: sha256:17d4d16bc218d5d686191edf72cc69a0d59b230d7a0c698dce915506d4ee30b5 size: 1165
```

Docker Regitry를 대신하는 기술

Harbor 사용

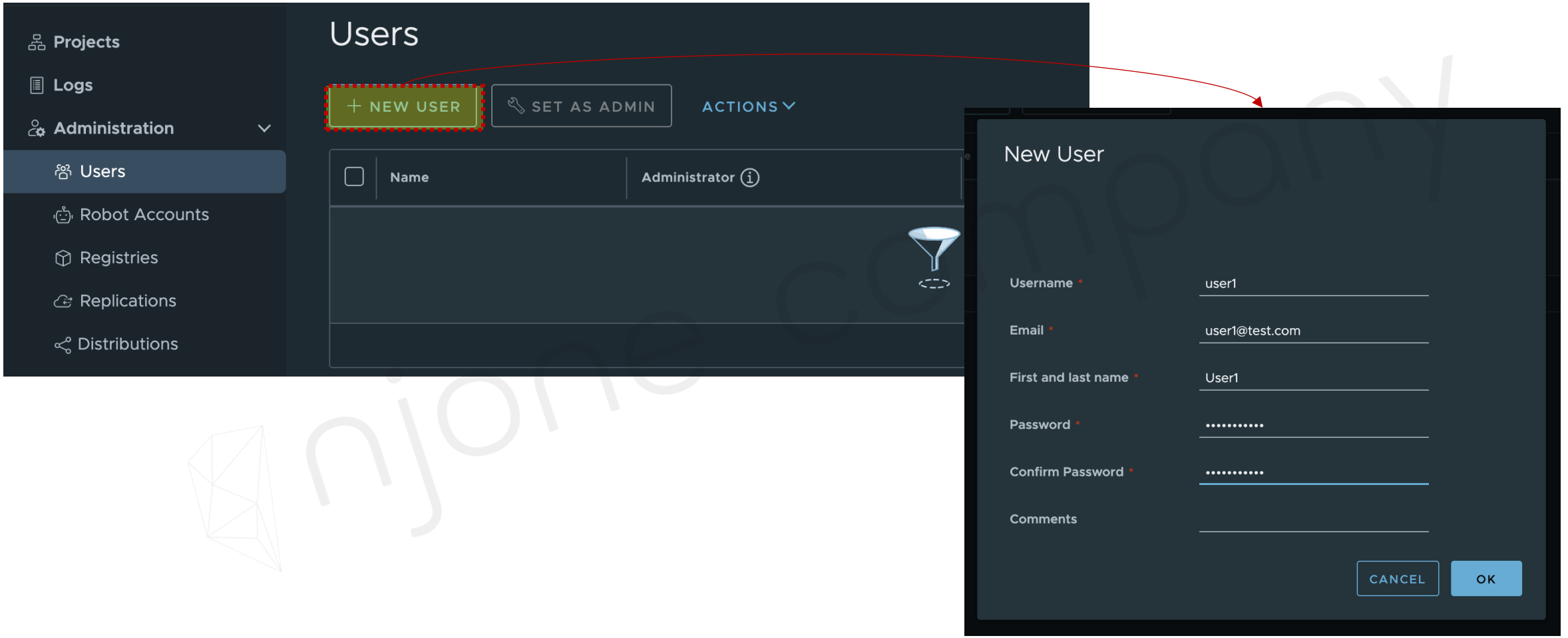
3) *Container#1]* docker push <IMAGE>



Docker Regitry를 대신하는 기술

Harbor 사용

4) *Container#2]* docker login –u user1 <Harbor IP address>



Docker Regitry를 대신하는 기술

Harbor 사용

4) *Container#2]* docker login –u user1 <Harbor IP address>

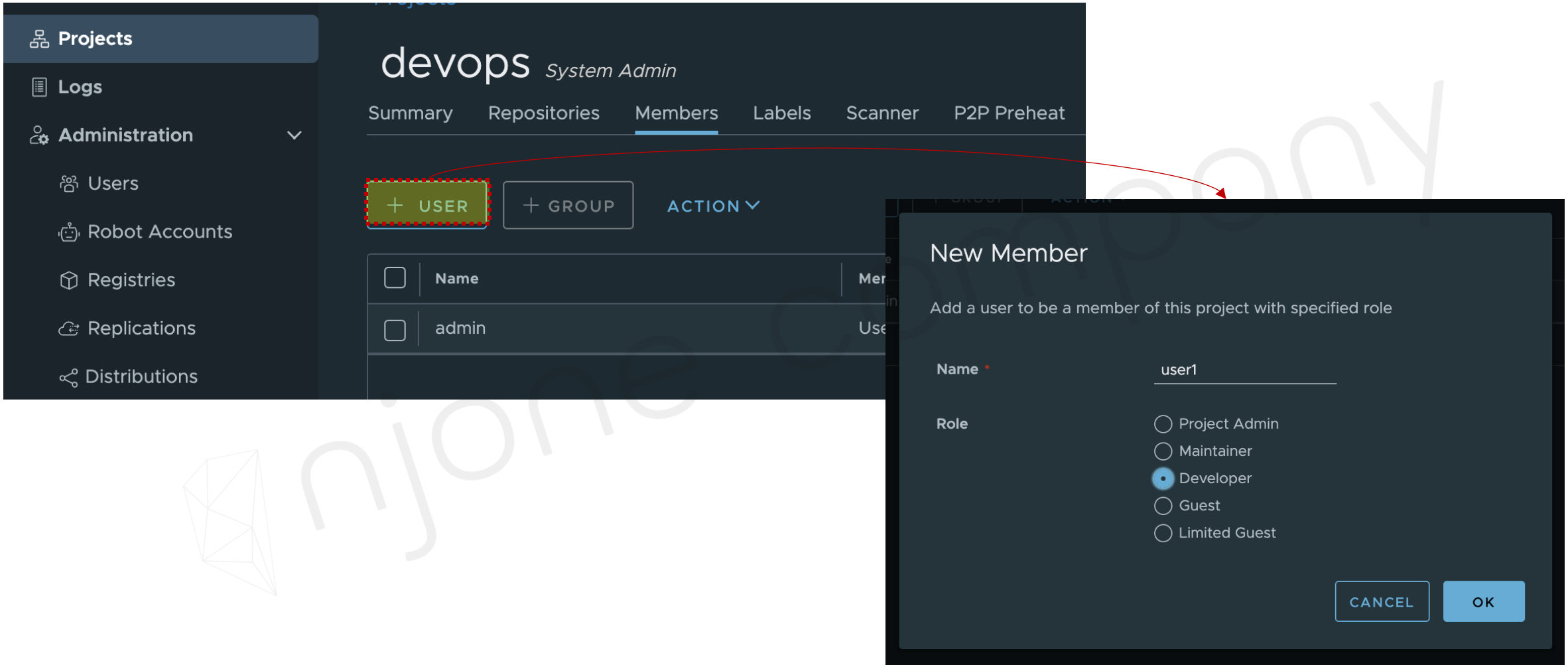
```
[root@33c30115eced ~]# docker login -u user1 172.30.11.76
Password:
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
[root@33c30115eced ~]#
```

```
[root@33c30115eced ~]# docker pull 172.30.11.76/devops/catalog-service:1.0
Error response from daemon: unauthorized: unauthorized to access repository: devops/catalog-service,
action: pull: unauthorized to access repository: devops/catalog-service, action: pull
[root@33c30115eced ~]#
```

Harbor 사용

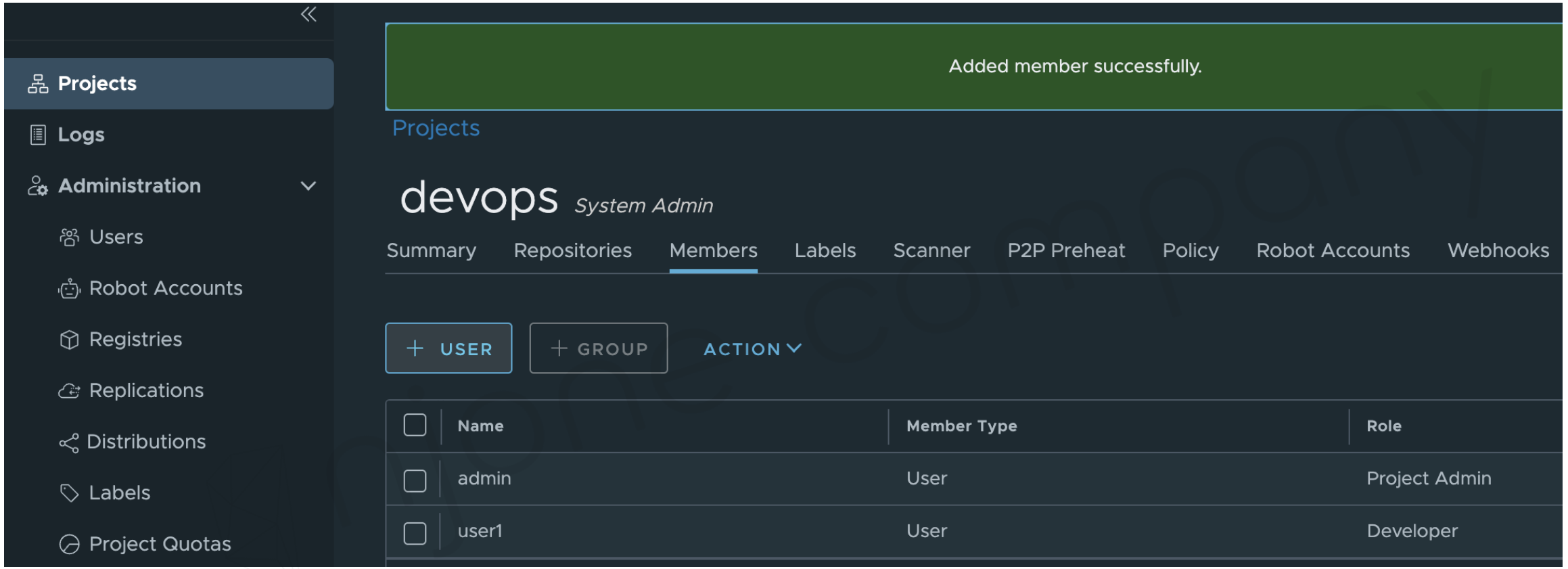
5) Container#2] docker pull <IMAGE>



Docker Regitry를 대신하는 기술

Harbor 사용

5) *Container#2]* docker pull <IMAGE>



Harbor 사용

5) *Container#2]* docker pull <IMAGE>

```
[root@33c30115eced ~]# docker pull 172.30.11.76/devops/catalog-service:1.0
1.0: Pulling from devops/catalog-service
45b42c59be33: Pull complete
a91c0c19c848: Pull complete
0a37071eae8e: Pull complete
17bec8782113: Pull complete
Digest: sha256:17d4d16bc218d5d686191edf72cc69a0d59b230d7a0c698dce915506d4ee30b5
Status: Downloaded newer image for 172.30.11.76/devops/catalog-service:1.0
172.30.11.76/devops/catalog-service:1.0
[root@33c30115eced ~]#
```

njone company